GoNautika Agency API

Version 2 — integration guide for API-enabled travel agencies

Andaman ferry ticketing · Port Blair · Swaraj Dweep · Shaheed Dweep

Search sailings, hold seats, issue tickets against your agency wallet, and manage them afterwards.

This guide is the companion to two machine-readable files shipped alongside it: an OpenAPI 3.0 specification (`gonautika-agency-api-v2.yaml`) and a Postman collection (`GoNautika-Agency-API-v2.postman_collection.json`). The specification is the reference for every field; this document explains the handful of things a reference cannot: how to sign a request, how to retry one safely, and who decides what anything costs.

> **THE ONE RULE WORTH READING TWICE**
>
> **You never send an amount.** No request in this API carries a fare, a fee, a trade discount or a refund figure — no such field exists, on any endpoint. Every charge is computed by GoNautika: a booking from the sailing's published fare and your agency's own trade discount rate; a cancellation and a reschedule from the charge basis frozen when the ticket was issued. This is why a preview and the action it previews cannot disagree.

The shape of the API

These conventions hold on every endpoint, so they are worth reading once here rather than rediscovering them one call at a time.

	How it works
Verbs and paths	Ordinary REST under `/api/v2`. `GET` reads, `POST` creates or acts, `DELETE` releases. Nothing that reads changes anything.
Status codes	Real ones — `201` for a booking, `402` when the wallet is short, `409` for a conflict. Branch on the status, not on the body.
Errors	RFC 7807 problem documents, sent as `application/problem+json`.
Credentials	A bearer token *and* a request signature. Both are checked on every call.
Dates and times	ISO throughout: `2026-09-01`, and `11:00:00` in India Standard Time. Each sailing also carries `departsAt` as an absolute instant with its offset, for clients that would rather not assume a zone.
Money	Indian rupees as plain JSON numbers. Fares exclude the passenger levy and the fuel surcharge, which are added per seat at booking.
Seat selection	Seats are held explicitly and a hold returns a *reservation id*. Bookings and reschedules are made against those ids, never against seat numbers.
Identifiers	UUIDs for sailings, seats, reservations and travellers. `bookingReference` is a 64-bit number carried as a **string**; PNRs look like `NK-49508JEK1T`.
After booking	A ticket can be read, cancelled in whole or in part, and moved to another sailing — each addressed by booking reference and PNR.

Credentials

GoNautika issues your agency two values together. They are not interchangeable, and both are needed on every request.

Value	What it does
API key	Identifies you. Sent as `Authorization: Bearer <apiKey>`. Treat it as a password — it authorises spending from your wallet.
Client id	Signs. It is the HMAC key for `X-Nautika-Signature` and is never transmitted. It survives key regeneration, so rotating the key does not change how you sign.

Keys are managed from the GoNautika back office and can be regenerated, which invalidates the previous one immediately. Because the two are separate, a leaked API key alone is not enough to spend your wallet: without the client id an attacker cannot produce a valid signature.

Signing a request

Every request carries three headers, and all three are checked.

Header	Value
`Authorization`	`Bearer <apiKey>`
`X-Nautika-Timestamp`	When you signed, as Unix epoch **seconds**
`X-Nautika-Signature`	HMAC-SHA256 of the string below, keyed with your client id, lower-case hex

The string you sign

```
{METHOD}\n{path and query}\n{timestamp}\n{body}
```

Four lines, joined by newlines:

- **Method**, upper-case: `GET`, `POST`, `DELETE`.
- **Path and query string**, exactly as transmitted — percent-encoding included. A space is `%20`, not a space. This is the single most common cause of a `401` during integration.
- **The timestamp digits**, character for character as they appear in the header.
- **The raw body**, byte for byte as sent. A request without a body signs an empty final line.

> **SIGN THE BYTES YOU SEND**
>
> Serialize your body *once*, then sign that string and transmit that same string. Re-serializing between signing and sending — a different key order, added whitespace, a pretty-printer — produces a different body and therefore a different signature, and the request is rejected. Every client in this guide is written that way for exactly this reason.

A worked example

A trip search for Port Blair to Swaraj Dweep on 1 September 2026, signed at epoch second 1788019124, signs this exact string — note `%20` in both island names, and the empty fourth line:

```
GET
/api/v2/trips?origin=Port%20Blair&destination=Swaraj%20Dweep&date=2026-09-01
1788019124
```

Which produces:

```
Authorization:        Bearer NT-xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
X-Nautika-Timestamp:  1788019124
X-Nautika-Signature:  5c8f1e3a...  (64 lower-case hex characters)
```

A client, in four languages

Each of these signs and sends a request, and each was run against a live gateway before being printed here. They differ only in the idioms of the language: one HMAC, three headers, and the discipline of sending the bytes that were signed.

> *Verified: every client below returned `200` from `GET /api/v2/agency` and `GET /api/v2/trips` against a running gateway.*

Java

```
import java.net.URI;
import java.net.http.*;
import java.nio.charset.StandardCharsets;
import java.time.Instant;
import javax.crypto.Mac;
import javax.crypto.spec.SecretKeySpec;

/** Signs and sends one GoNautika Agency API v2 request. */
public class GoNautikaClient {

    private final String baseUrl, apiKey, clientId;
    private final HttpClient http = HttpClient.newHttpClient();

    public GoNautikaClient(String baseUrl, String apiKey, String clientId) {
        this.baseUrl = baseUrl;
        this.apiKey = apiKey;
        this.clientId = clientId;
    }

    /**
     * @param pathAndQuery e.g. "/api/v2/trips?origin=Port%20Blair&date=2026-09-01" — already
     *                     percent-encoded, because that is what the server signs.
     * @param body         the exact JSON to send, or null for a request without one.
     */
    public HttpResponse<String> send(String method, String pathAndQuery, String body, String idempotencyKey)
            throws Exception {

        String raw = body == null ? "" : body;
        long timestamp = Instant.now().getEpochSecond();
        String payload = method.toUpperCase() + "\n" + pathAndQuery + "\n" + timestamp + "\n" + raw;

        HttpRequest.Builder request = HttpRequest.newBuilder(URI.create(baseUrl + pathAndQuery))
                .header("Authorization", "Bearer " + apiKey)
                .header("X-Nautika-Timestamp", Long.toString(timestamp))
                .header("X-Nautika-Signature", hmacSha256Hex(payload, clientId));

        if (idempotencyKey != null) request.header("Idempotency-Key", idempotencyKey);

        if (body == null) {
            request.method(method, HttpRequest.BodyPublishers.noBody());
        } else {
            // Send exactly the bytes that were signed — re-serializing here would break the signature.
            request.header("Content-Type", "application/json")
                   .method(method, HttpRequest.BodyPublishers.ofString(raw, StandardCharsets.UTF_8));
        }

        return http.send(request.build(), HttpResponse.BodyHandlers.ofString());
    }

    private static String hmacSha256Hex(String payload, String key) throws Exception {
        Mac mac = Mac.getInstance("HmacSHA256");
        mac.init(new SecretKeySpec(key.getBytes(StandardCharsets.UTF_8), "HmacSHA256"));
        StringBuilder hex = new StringBuilder(64);
        for (byte b : mac.doFinal(payload.getBytes(StandardCharsets.UTF_8))) {
            hex.append(String.format("%02x", b));
        }
        return hex.toString();
    }
}
```

C#

```
using System.Globalization;
using System.Security.Cryptography;
using System.Text;

/// <summary>Signs and sends GoNautika Agency API v2 requests.</summary>
public sealed class GoNautikaClient(string baseUrl, string apiKey, string clientId)
{
    private readonly HttpClient _http = new() { BaseAddress = new Uri(baseUrl) };

    /// <param name="pathAndQuery">
    /// e.g. "/api/v2/trips?origin=Port%20Blair&amp;date=2026-09-01" — already percent-encoded,
    /// because that is what the server signs.
    /// </param>
    /// <param name="body">The exact JSON to send, or null for a request without one.</param>
    public async Task<(int Status, string Body)> SendAsync(
        HttpMethod method, string pathAndQuery, string? body = null, string? idempotencyKey = null)
    {
        var raw = body ?? string.Empty;
        var timestamp = DateTimeOffset.UtcNow.ToUnixTimeSeconds();
        var payload = $"{method.Method.ToUpperInvariant()}\n{pathAndQuery}\n{timestamp}\n{raw}";

        using var request = new HttpRequestMessage(method, pathAndQuery);
        request.Headers.TryAddWithoutValidation("Authorization", $"Bearer {apiKey}");
        request.Headers.TryAddWithoutValidation("X-Nautika-Timestamp",
            timestamp.ToString(CultureInfo.InvariantCulture));
        request.Headers.TryAddWithoutValidation("X-Nautika-Signature", Sign(payload, clientId));

        if (idempotencyKey is not null)
        {
            request.Headers.TryAddWithoutValidation("Idempotency-Key", idempotencyKey);
        }

        if (body is not null)
        {
            // Send exactly the bytes that were signed — re-serializing here would break the signature.
            request.Content = new StringContent(raw, Encoding.UTF8, "application/json");
        }

        using var response = await _http.SendAsync(request);
        return ((int)response.StatusCode, await response.Content.ReadAsStringAsync());
    }

    private static string Sign(string payload, string key)
    {
        using var hmac = new HMACSHA256(Encoding.UTF8.GetBytes(key));
        return Convert.ToHexStringLower(hmac.ComputeHash(Encoding.UTF8.GetBytes(payload)));
    }
}
```

Python

```
import hashlib, hmac, json, time, urllib.error, urllib.request


class GoNautikaClient:
    """Signs and sends GoNautika Agency API v2 requests."""

    def __init__(self, base_url, api_key, client_id):
        self.base_url = base_url.rstrip("/")
        self.api_key = api_key
        self.client_id = client_id

    def send(self, method, path_and_query, body=None, idempotency_key=None):
        """
        `path_and_query` is already percent-encoded — e.g.
        "/api/v2/trips?origin=Port%20Blair&date=2026-09-01" — because that is what the server signs.
        `body` is serialized once, then both signed and sent as those exact bytes.
        """
        raw = "" if body is None else json.dumps(body, separators=(",", ":"))
        timestamp = str(int(time.time()))
        payload = f"{method.upper()}\n{path_and_query}\n{timestamp}\n{raw}"

        headers = {
            "Authorization": f"Bearer {self.api_key}",
            "X-Nautika-Timestamp": timestamp,
            "X-Nautika-Signature": hmac.new(
                self.client_id.encode(), payload.encode(), hashlib.sha256).hexdigest(),
        }
        if body is not None:
            headers["Content-Type"] = "application/json"
        if idempotency_key:
            headers["Idempotency-Key"] = idempotency_key

        request = urllib.request.Request(
            self.base_url + path_and_query,
            data=raw.encode() if body is not None else None,
            headers=headers,
            method=method.upper(),
        )

        try:
            with urllib.request.urlopen(request) as response:
                return response.status, json.loads(response.read() or b"null")
        except urllib.error.HTTPError as error:
            # A rejection is an answer, not a transport failure: read the problem document.
            return error.code, json.loads(error.read() or b"null")
```

Node.js

```
import { createHmac, randomUUID } from 'node:crypto';

/** Signs and sends GoNautika Agency API v2 requests. */
export class GoNautikaClient {
  constructor(baseUrl, apiKey, clientId) {
    this.baseUrl = baseUrl.replace(/\/$/, '');
    this.apiKey = apiKey;
    this.clientId = clientId;
  }

  /**
   * @param {string} pathAndQuery e.g. '/api/v2/trips?origin=Port%20Blair&date=2026-09-01' —
   *   already percent-encoded, because that is what the server signs.
   * @param {object} [body] serialized once, then both signed and sent as those exact bytes.
   */
  async send(method, pathAndQuery, body, idempotencyKey) {
    const raw = body === undefined ? '' : JSON.stringify(body);
    const timestamp = Math.floor(Date.now() / 1000).toString();
    const payload = `${method.toUpperCase()}\n${pathAndQuery}\n${timestamp}\n${raw}`;

    const headers = {
      Authorization: `Bearer ${this.apiKey}`,
      'X-Nautika-Timestamp': timestamp,
      'X-Nautika-Signature': createHmac('sha256', this.clientId).update(payload).digest('hex'),
    };
    if (body !== undefined) headers['Content-Type'] = 'application/json';
    if (idempotencyKey) headers['Idempotency-Key'] = idempotencyKey;

    const response = await fetch(this.baseUrl + pathAndQuery, {
      method: method.toUpperCase(),
      headers,
      // The signed string, not a re-serialization of it.
      body: body === undefined ? undefined : raw,
    });

    const text = await response.text();
    return { status: response.status, body: text ? JSON.parse(text) : null };
  }
}
```

The replay window

A timestamp must be within **300 seconds** of server time. Outside that, the response is a `401` that states the measured difference — unlike every other authentication failure, which is deliberately opaque:

```
{
  "title":  "Unauthorized",
  "status": 401,
  "detail": "The request timestamp is -788018402s from server time, outside the
             permitted 300s window. Check the sending system's clock."
}
```

The window is public, so saying this discloses nothing — and without it, an integrator with a drifting clock has nothing to work from.

The clock-offset allowance

A timestamp out by exactly one whole timezone offset — India's 5h30m, 19800 seconds — is also accepted, in either direction. A Unix timestamp is an absolute instant, so a correct client and a correct server agree whatever zone each runs in; in practice a client that takes its local wall clock and encodes it as though it were UTC is out by precisely its own offset, and these servers run in India Standard Time while callers commonly run in UTC.

This does *not* widen the window. Three separate windows are accepted, each still 300 seconds wide, so any one captured request ages out exactly as fast. A timestamp *between* the windows is refused. A request accepted this way is logged as a warning at our end, so a skewed client stays visible rather than being silently normalised — fix your clock rather than relying on it.

One signature, one request

A verified signature is remembered. Sending the **same signature again** answers:

```
409 Conflict
{
  "title":  "Request already processed",
  "status": 409,
  "detail": "This exact request has already been received. Sign a new request
             rather than resending one."
}
```

The signature covers the method, path, timestamp and body together, which makes it the identity of that exact request. Reads are exempt: `GET`, `HEAD` and `OPTIONS` are not replay-guarded, because a timestamp has one-second resolution and two identical reads in the same second are ordinary traffic.

> **THIS IS WHAT "RETRY" MEANS HERE**
>
> **To retry, re-sign.** Resending the identical bytes is not a retry — it is a replay, and it answers `409`. Take a fresh timestamp, compute a fresh signature, send the same body.

Retrying a booking safely

`POST /api/v2/bookings` requires an `Idempotency-Key` header — any value unique to the attempt, at most 128 characters; a UUID is the obvious choice. Without one the request is refused outright, because a client whose response

is lost to a timeout cannot tell a booking that failed from one that succeeded, and retrying blind would book and charge twice.

The first request under a key runs and its outcome is remembered for 24 hours. A later request under the same key returns that same outcome — the same booking reference, the same PNRs, the same status — instead of booking again.

The two halves fit together like this

1. Send the booking with a fresh `Idempotency-Key`, a timestamp, and a signature.
2. The response is lost — a timeout, a dropped connection, a crashed process. You do not know whether it worked.
3. Retry: **a new timestamp and a new signature**, the **same** `Idempotency-Key`, the same body.
4. You get the original outcome back. If the first attempt booked, you receive that booking; if it failed, you receive that failure. Either way nothing happened twice.

> **DO NOT REUSE THE SIGNATURE**
>
> Resending the byte-identical request — same timestamp, same signature — trips the replay guard and answers `409 Request already processed` *before* the stored outcome can be returned. The idempotency key is on the request; the replay guard is on the signature. A correct retry changes the signature and keeps the key.

What the other conflicts mean

Response	Cause	What to do
`409` Idempotency key reused	The same key was sent with a different body.	Use a new key. A key identifies one attempt at one request.
`409` Request in progress	An earlier request under this key is still running.	Wait and retry the same key; do not start a second booking.
`409` Request already processed	The same *signature* arrived twice.	Re-sign with a fresh timestamp.

A request that throws records nothing, leaving the key free to retry. Keys are scoped to your agency, so two agencies choosing the same value never collide.

Errors

Failures are RFC 7807 problem documents, sent as `application/problem+json`:

```
{
  "type":    "https://tools.ietf.org/html/rfc9110#section-15.5.5",
  "title":   "Not found",
  "status":  404,
  "detail":  "No booking 881566206208159744.",
  "traceId": "00-b0e07444818c5318194bb7d5e07d9d72-c005255199173834-00"
}
```

Branch on `status`. `detail` is written for a person reading a log and may be reworded; `traceId` is what to quote to GoNautika support.

Status	Meaning	Is retrying worth it?
`400`	Malformed, or self-contradictory — passengers that do not match the reservations, for instance.	No. Fix the request.
`401`	The key, the timestamp or the signature did not check out.	Only if the timestamp was stale.
`402`	The wallet and its credit line cannot cover the charge. **Nothing was created or charged.**	After a top-up.
`403`	Your agency is not permitted this action.	No — settled in the back office.
`404`	No such thing, **or it exists and is not yours**.	No.
`409`	A conflict: seats gone, a hold lapsed, a replayed signature, a reused key.	Depends — see above.
`503`	A GoNautika service is temporarily unreachable.	Yes, shortly, with a fresh signature.

> **WHY ANOTHER AGENCY'S BOOKING IS A 404**
>
> Asking about a booking that exists but belongs to someone else answers `404`, not `403`. Distinguishing the two would let a caller sweep the reference space and learn which references are real. The same reasoning covers PNRs, travellers and seat holds: ids that are not yours are reported as though they do not exist. Verified: a second agency asking about the booking in this guide received `404 No booking 881566206208159744.`

Booking a ticket

Four calls, in order. Everything below is a real exchange with a live gateway.

1. **Search** — `GET /api/v2/trips` for the sailings on a date, with fares and the state of every seat.
2. **Hold** — `POST /api/v2/seat-holds` for the seats you want. You get a *reservation id* per seat.
3. **Book** — `POST /api/v2/bookings` against those reservation ids. The wallet is charged in the same transaction.
4. **Read back** — `GET /api/v2/bookings/{reference}` whenever you need the tickets and traveller ids.

1 · Search

```
GET /api/v2/trips?origin=Port%20Blair&destination=Swaraj%20Dweep&date=2026-09-01
```

```
[
  {
    "tripId": "4ed0fae4-5ab8-49f4-ab32-63140b57ccf4",
    "origin": "Port Blair", "destination": "Swaraj Dweep",
    "tripDate": "2026-09-01",
    "departureTime": "11:00:00", "arrivalTime": "12:30:00",
    "departsAt": "2026-09-01T11:00:00+05:30",
    "vesselId": "39185d0c-f232-46fb-b612-652da8c5c4de", "vesselName": "Nautika",
    "seatClasses": [
      { "seatClassName": "Royal",  "baseFare": 2100, "infantFare": 0,
        "availableSeatCount": 82,
        "seats": [ { "seatId": "c9fe9324-...", "seatNumber": "1A", "deck": "U",
                     "isAvailable": true, "isBooked": false, "isBlocked": false } ] },
      { "seatClassName": "Luxury", "baseFare": 1900, "infantFare": 0,
        "availableSeatCount": 173, "seats": [ ... ] }
    ]
  }
]
```

Only seats with `isAvailable: true` can be held. `isBooked` covers seats that are sold *and* seats permanently out of service; `isBlocked` covers seats set aside and seats under someone's checkout hold — possibly your own.

> **AN EMPTY LIST IS A NORMAL ANSWER**
>
> Sailings departing inside the notice period are withheld, and a sailing with no bookable class is omitted entirely. A seat class absent from `seatClasses` is not on sale on that sailing, whatever the vessel's layout suggests — availability is published per sailing, not derived from the boat. Treat `[]` as "nothing to sell", never as an error.

2 · Hold

```
POST /api/v2/seat-holds

{ "tripId": "4ed0fae4-5ab8-49f4-ab32-63140b57ccf4",
  "seatIds": [ "a608af16-001a-4ea9-a999-cc549260ee57",
               "36d2c4f6-a6b0-4799-8fcc-bf731476922a" ] }
```

```
201 Created

{ "tripId": "4ed0fae4-5ab8-49f4-ab32-63140b57ccf4",
  "expiresAt": "2026-08-29T21:34:22.013139+05:30",
  "seats": [
    { "reservationId": "8e452630-214c-40e5-ad23-856706a137ff",
      "seatId": "36d2c4f6-...", "seatNumber": "1F", "isSelected": true },
    { "reservationId": "70d74f53-ae94-461e-b576-956cdd3642ac",
      "seatId": "a608af16-...", "seatNumber": "1E", "isSelected": true }
  ] }
```

A booking is made against **reservation ids**, not against seats, so this step is not optional — there is no implicit-hold path.

Either every seat asked for is held or none is. If one has gone since the search, the rest are released and the request is refused with `409`: holding half a party's seats helps nobody. A hold lapses on its own after about fifteen minutes — `expiresAt` is authoritative — so an abandoned checkout never strands a seat. `DELETE /api/v2/seat-holds` returns them to sale immediately, which is worth calling when a customer walks away.

3 · Book

```
POST /api/v2/bookings
Idempotency-Key: 8f14e45f-ceea-467a-9f0d-1c2b3a4d5e6f

{ "reservationIds": [ "8e452630-214c-40e5-ad23-856706a137ff",
                      "70d74f53-ae94-461e-b576-956cdd3642ac" ],
  "passengers": [
    { "reservationId": "8e452630-...", "firstName": "Anita", "lastName": "Sharma",
      "gender": "F", "age": 34, "nationality": "India" },
    { "reservationId": "70d74f53-...", "firstName": "Rahul", "lastName": "Sharma",
      "gender": "M", "age": 36, "nationality": "India" }
  ],
  "contact": { "email": "anita.sharma@example.com", "phone": "9000000001" },
  "remarks": "Honeymoon package" }
```

```
201 Created

{ "bookingReference": "881566206208159744",
  "status": "CMPL",
  "tickets": [ { "pnr": "NK-49508JEK1T",
                 "tripId": "4ed0fae4-5ab8-49f4-ab32-63140b57ccf4",
                 "status": "CMPL" } ],
  "charge": { "totalFare": 3800,
              "totalPassengerLevy": 100,
              "totalFuelSurcharge": 700,
              "tradeDiscountAmount": 570,
              "amountCharged": 4030 } }
```

> *Verified live. The agency wallet moved from ₹56,280 to ₹52,250 — exactly the ₹4,030 reported.*

Two rules govern what you may send:

- **Every reservation must still be held, and held by you.** A hold that has lapsed, has already been booked, or belongs to another agency refuses the whole booking. Ownership is checked at our end; you cannot book against someone else's hold even knowing its id.
- **Passengers and reservations must be exactly the same set.** Each seated traveller names the reservation it occupies; an infant names none, because it takes no seat. A seat cannot be paid for and left empty, nor a traveller left without one.

You receive **one ticket per seat class**. A party split across Royal and Luxury comes back as two PNRs under one booking reference — each a ticket in its own right, cancellable and reschedulable separately.

Via itineraries are not sold here

A *via* itinerary is a single PNR spanning two sailings — boarding at Port Blair, calling at Swaraj Dweep and travelling on to Shaheed Dweep, keeping the same physical seats throughout, and priced at one through fare rather than as two tickets.

> **THIS API SELLS DIRECT SAILINGS ONLY**
>
> `GET /api/v2/trips` returns direct sailings and reports only their direct fares. A sailing's through fares to a further island are not listed, and there is no way to book against one. So a search for Port Blair to Shaheed Dweep answers `[]` on a date when that journey is perfectly sellable through a Port Blair–Swaraj Dweep sailing, because such a sailing is offered here only as far as Swaraj Dweep.

To sell the same journey through this API, book the two sailings as **two separate bookings** — Port Blair to Swaraj Dweep, then Swaraj Dweep to Shaheed Dweep. Be clear with your customer about what that changes:

- **Two fares, not one.** Each leg is priced at its own direct fare, and the levy and the fuel surcharge are charged per seat on *each*. A through fare is normally cheaper.
- **Two booking references and two PNRs**, cancelled and rescheduled independently. Nothing links them, so cancelling one leaves the other travelling.
- **Seats are chosen separately** on each sailing, and need not be the same ones.

Where the through fare matters, the itinerary has to be booked through the agency web application. A via ticket booked that way still belongs to your agency, so it is readable here — it carries `"isVia": true` — and it can be cancelled here. It cannot be rescheduled here; see *Rescheduling* below.

> **BOOKINGREFERENCE IS A STRING**
>
> It is a 64-bit number, carried as a string precisely because it does not survive being parsed into a double-precision float. Keep it as text end to end — JavaScript in particular will silently corrupt it.

What a booking costs, and who worked it out

Look again at the request above: there is no fare in it, no total, no trade discount. There is nowhere to put one. The gateway prices the booking itself:

Component	Where it comes from	In this booking
Base fare	The sailing's *published* fare for that seat class, read at booking time	2 × ₹1,900 = ₹3,800
Passenger levy	A fixed amount per seat	2 × ₹50 = ₹100
Fuel surcharge	A fixed amount per seat	2 × ₹350 = ₹700
Trade discount	Your agency's API trade discount rate, applied to the **base fare only**	15% of ₹3,800 = ₹570
Charged	Gross total less the trade discount	**₹4,030**

The trade discount is never taken on the levy or the surcharge. The rate is the one configured for your agency on the API channel — it is not the same figure as your web-channel rate, and it is not something a request can influence.

One refusal is worth knowing about in advance, because it is not fixed by retrying: an agency with **no API trade discount rate configured** is refused with `403`. That is treated as a misconfiguration rather than as a booking at full price. It is settled with GoNautika, not in your client.

> **A 402 COSTS NOTHING**
>
> The booking and the wallet debit are one transaction. `402 Payment Required` means **nothing was created and nothing was charged** — no seats booked, no partial state, no compensating call to make. Top up and try again.

4 · Read the booking back

```
GET /api/v2/bookings/881566206208159744
```


```
{ "bookingReference": "881566206208159744", "status": "CMPL",
  "origin": "Port Blair", "destination": "Swaraj Dweep",
  "bookedAt": "2026-08-29T21:19:22.222321",
  "tickets": [ {
    "pnr": "NK-49508JEK1T", "status": "CMPL",
```

```
    "tripId": "4ed0fae4-...", "tripDate": "2026-09-01",
    "departureTime": "11:00:00", "arrivalTime": "12:30:00",
    "vesselName": "Nautika", "seatClassName": "Luxury", "isVia": false,
    "passengers": [
      { "passengerId": "8e47aa92-9375-4ec4-a2ad-a0e7af65a579",
        "firstName": "Anita", "lastName": "Sharma", "gender": "F", "age": 34,
        "isInfant": false, "isLead": false, "isCancelled": false, "seatNumber": "1F" },
      { "passengerId": "aace1da2-f095-4907-b5bd-b35341a73d48",
        "firstName": "Rahul", "lastName": "Sharma", "gender": "M", "age": 36,
        "isInfant": false, "isLead": true,  "isCancelled": false, "seatNumber": "1E" }
    ] } ] }
```


This is where `passengerId` comes from — the id needed to cancel one traveller rather than the whole ticket. Ticket `status` reads `CMPL` complete, `PRTL` partly cancelled, `CNCL` cancelled, `RSCD` rescheduled.

After the ticket is issued

Everything here is addressed by **booking reference and PNR** — never by an internal id — and every operation confirms the booking is yours before it does anything.

Operation	Endpoint
What cancelling the ticket would refund	`GET .../tickets/{pnr}/cancellation-preview`
Cancel the whole ticket	`POST .../tickets/{pnr}/cancellation`
What cancelling one traveller would refund	`GET .../passengers/{id}/cancellation-preview`
Cancel one traveller	`POST .../passengers/{id}/cancellation`
Move the ticket to another sailing	`POST .../tickets/{pnr}/reschedule`

Previews are the same calculation as the action

A preview is not a quote you send back for confirmation. It runs the identical calculation, in the identical place, against the charge basis frozen when the ticket was issued. There is nothing to echo and nothing to get out of step: quote the preview to your customer, then act, and the figures agree.

The only thing that changes between the two is the clock — the charge depends on how close departure is — so preview immediately before confirming.

Cancelling one traveller

```
GET /api/v2/bookings/881566206208159744/tickets/NK-49508JEK1T
        /passengers/8e47aa92-9375-4ec4-a2ad-a0e7af65a579/cancellation-preview
```

```
{ "totalFare": 2300,
  "tradeDiscountAmount": 285,
  "netPaymentReceived": 2015,
  "refundAmount": 1915,
  "cancellationCharge": 100,
  "hoursBeforeDeparture": 61.67,
  "passengers": [ { "fullName": "Anita Sharma", "tripFare": 1900,
                    "fuelSurchargeAndLevy": 400, "total": 2300,
                    "refundAmount": 1915 } ] }
```

Then the cancellation itself, which carries a reason and nothing else:

```
POST .../passengers/8e47aa92-.../cancellation

{ "remarks": "Traveller withdrew from the party" }
```

> *Verified live: the response was identical to the preview — ₹1,915 refunded, ₹100 charge — and the wallet rose by exactly ₹1,915. The ticket became `PRTL`; Rahul kept travelling on the same PNR.*

Only that traveller's seat is released. Cancelling someone already cancelled answers `409`; naming a traveller who is on a different ticket answers `404`, checked before anything is sent onward.

Cancelling the whole ticket

The same shape, addressed at the PNR. For this booking the preview quoted:

```
{ "totalFare": 4600, "tradeDiscountAmount": 570,
  "netPaymentReceived": 4030,
  "refundAmount": 3830, "cancellationCharge": 200,
  "hoursBeforeDeparture": 61.67,
  "passengers": [ { "fullName": "Anita Sharma", ..., "refundAmount": 1915 },
                  { "fullName": "Rahul Sharma", ..., "refundAmount": 1915 } ] }
```

Note that the charge is ₹200 for two travellers, not ₹200 for the ticket: the tier is applied per head. `hoursBeforeDeparture` goes negative once the sailing has left — the preview will still answer, the cancellation will not.

Cancelling a via ticket

A via ticket cancels like any other, in whole or one traveller at a time. Send only a reason: the seats are released on **both** of its sailings automatically, and the refund covers the whole through journey. There is nothing extra to name and no second call to make.

This is also how a via journey is moved — cancelled and rebooked — because it cannot be rescheduled.

The cancellation tiers

When you cancel	What is refunded
More than 48 hours before departure	Everything taken, less a flat fee per traveller
Between 24 and 48 hours	Half the trip fare
Within 24 hours	Nothing

These thresholds and amounts are configuration and can change. Never reimplement them: read `cancellationCharge` and `refundAmount` from the preview, which is authoritative by construction.

Rescheduling

A reschedule moves every still-active traveller on a ticket to another sailing and re-prices them there. It takes **reservation ids, not seats** — hold the seats on the destination sailing first, exactly as for a booking, and they are checked to be yours the same way.

1. Search the destination date and pick a sailing.
2. `POST /api/v2/seat-holds` for as many seats as there are active travellers, all on that one sailing.
3. `POST .../tickets/{pnr}/reschedule` with those reservation ids and a reason.

```
POST /api/v2/bookings/881566206208159744/tickets/NK-49508JEK1T/reschedule

{ "reservationIds": [ "621fa80e-dbab-4c88-843a-e361d09943d2" ],
  "remarks": "Customer moved to the morning sailing" }
```

```
200 OK

{ "pnr": "NK-49508JEK1T",
  "tripId": "b4a4d57b-53f5-4dba-9b77-ba47b06808b2",
```

```
  "fareDifference": -650,
  "rescheduleFee": 100,
  "amountCharged": 100 }
```

Verified live: one traveller moved from a ₹1,900 sailing to a ₹1,250 one. The ₹650 saving was not returned; the ₹100 fee was charged; the wallet fell by exactly ₹100.

Again, no figure was sent. The fare difference is computed against the new sailing's published fare, and the fee is added — **per traveller moved**, infants excepted, not a flat charge per ticket. A ticket for four costs four fees.

A FARE DECREASE IS NEVER REFUNDED

`amountCharged` is `max(fareDifference, 0) + rescheduleFee`. Moving to a cheaper sailing still costs the fee and returns nothing. `fareDifference` is reported signed so you can show a customer what happened, but do not treat a negative value as a credit.

A VIA TICKET CANNOT BE RESCHEDULED

A ticket carrying `"isVia": true` — one PNR travelling two sailings — is refused with `409`:

```
{ "title":  "Conflict",
  "status": 409,
  "detail": "This is a via ticket, travelling two sailings on one PNR, and a via
             ticket cannot be rescheduled. Cancel it and book the new journey
             instead." }
```

The refusal is on what the ticket *is*, so it comes back before any hold you named is even looked at — check `isVia` before taking holds, or you will have paid for seats you cannot use. **Cancel the ticket and book the new journey instead**: the cancellation refunds the whole through journey under the ordinary tiers and releases the seats on both sailings.

The `onwardReservationIds` field exists for deployments where GoNautika has enabled via rescheduling. Unless you have been told it is on for yours, it is inert — sending it does not lift the refusal. Where it *is* enabled it becomes required for a via ticket, and must name holds covering **the same seats** as the first sailing's, because a via ticket keeps its seats across both.

Two further limits to design around

THERE IS NO RESCHEDULE PREVIEW

The underlying booking system offers none, so neither does this API — quoting a figure the system could not stand behind would be worse than not quoting one. To tell a customer the cost beforehand, read the published fares for both sailings from `GET /api/v2/trips`, take the difference if positive, and add the fee per non-infant traveller. Treat that as an estimate: the authoritative figure is the one the reschedule returns.

A RESCHEDULED TICKET CANNOT THEN BE CANCELLED

Once a ticket is `RSCD`, cancelling it needs a permission the API channel does not carry, and the attempt answers:

```
403 Not permitted
"This leg has already been rescheduled and cannot be cancelled without the
 CAN_CANCEL_RESCHEDULED_TICKET permission."
```

The cancellation *preview* still answers `200` for such a ticket and quotes a refund — so do not treat a successful preview as proof that the cancellation will go through. If a customer may still cancel, cancel first and rebook rather than rescheduling. A rescheduled ticket that must be cancelled has to go through GoNautika.

Where every charge is decided

Collected in one place, because it is the question integrators ask most.

Operation	What your request carries	Who computes the money
Booking	Reservation ids, travellers, contact details, an optional note	The gateway — published fare per seat, levy and surcharge per seat, your API trade discount rate on the base fare
Cancel a ticket	A reason	The booking system — from the frozen charge basis and the tier at that moment
Cancel one traveller	A reason	The booking system — the same calculation, that traveller's share
Reschedule	The new sailing's reservation ids, a reason	The booking system — re-priced on the new sailing, plus the per-traveller fee

Three consequences follow, and they are the reason it is built this way:

- **A preview and its action cannot disagree.** They are one calculation, not a quote passed back and forth for confirmation.
- **An integration cannot mis-price a booking.** There is no field through which a wrong number could arrive, so no client bug can under- or over-charge a wallet.
- **Fares can change without breaking anyone.** Pricing is read at the moment of action, so a fare revision needs no client release.

What you should do with the figures you receive is show them and record them. What you should not do is recompute them: reimplementing the tiers or the trade discount arithmetic in your own client puts you one configuration change away from telling a customer the wrong number.

Endpoint reference

Method & path	What it does	Success
`GET /api/v2/agency`	The calling agency and its funds	`200`
`GET /api/v2/trips`	Sailings on a date, with fares and seats	`200`
`POST /api/v2/seat-holds`	Hold seats → reservation ids	`201`
`DELETE /api/v2/seat-holds`	Release your own holds	`204`
`POST /api/v2/bookings`	Book the held seats, charge the wallet	`201`
`GET /api/v2/bookings/{ref}`	One of your bookings, with travellers	`200`
`GET .../tickets/{pnr}/cancellation-preview`	What cancelling would refund	`200`
`POST .../tickets/{pnr}/cancellation`	Cancel the whole ticket	`200`
`GET .../passengers/{id}/cancellation-preview`	What cancelling one traveller would refund	`200`
`POST .../passengers/{id}/cancellation`	Cancel one traveller	`200`
`POST .../tickets/{pnr}/reschedule`	Move the ticket to another sailing	`200`

Paths under `.../tickets/` and `.../passengers/` are relative to `/api/v2/bookings/{bookingReference}`. Every one requires all three authentication headers; `POST /api/v2/bookings` additionally requires `Idempotency-Key`.

A checklist before you go live

- Percent-encode the path and query *before* signing, and sign the same string you send.
- Serialize the body once; sign and transmit those exact bytes.
- Send the timestamp in **seconds**, not milliseconds.
- Keep the sending system's clock in step — and do not rely on the timezone allowance to cover a genuine drift.
- Generate a fresh `Idempotency-Key` per booking attempt; reuse it, with a **fresh signature**, when retrying that same attempt.
- Never resend a signed request byte for byte — re-sign instead.
- Carry `bookingReference` as a string throughout.
- Treat an empty trip list as normal, and a `404` as "not yours or not there".
- Preview immediately before cancelling, and show the customer the previewed figures.
- Check `isVia` before taking holds for a reschedule — a via ticket is refused, and you would be holding seats you cannot use.
- Sell a through journey as two direct bookings, and tell the customer it is two fares and two PNRs.
- Release holds you no longer need rather than letting them lapse.
- Do not reimplement fares, the trade discount, cancellation tiers or the reschedule fee.

Getting help

Quote the `traceId` from the problem document — it identifies the exact request in our logs. For anything about credentials, trade discount rates or permissions, contact GoNautika directly; none of those can be changed through this API.